WARLOCK PROJECT / COLLABORATION SUMMARY

What we built together

A record of the coding editor, live dual-player challenge mode, code checking, and deployment setup developed through our work.

1	2	80 / 20
supported language	players per room	correctness / efficiency

The result

Warlock grew from a browser-based code editor into a multiplayer coding challenge experience: players can join a room, work in parallel, run JavaScript solutions against challenge cases, and compare results.

A clear split of responsibilities

Vercel serves the frontend. **Render** runs the room and WebSocket server. JavaScript challenge solutions run in a short-lived browser Web Worker.

This summary covers the implemented project work and its current browser-based evaluation flow.

FEATURES / IMPLEMENTED WORK

From editor to head-to-head coding

The main product work centered on making JavaScript coding challenges playable by two people at once.

01 JavaScript editor

Focused the challenge editor on JavaScript and added starter templates for the available problems.

02 Live dual-player rooms

Built room creation and joining with short room codes, player names, shared challenge state, and live code updates between two players.

03 Submit and compare

Added individual submissions, test-case results, score comparison, and winner selection. The submit control and dual-editor layout were adjusted so the action remains visible.

04 Correctness plus efficiency

The score weights test correctness at 80% and a JavaScript efficiency heuristic at 20%. The heuristic provides guidance; it is not a formal complexity proof.

05 Challenge evaluation

Runs submitted JavaScript against the challenge cases in a short-lived Web Worker.

06 Execution timeout

Added a worker timeout so an infinite loop cannot keep the evaluation running indefinitely.

Product decisions and fixes

- Kept the frontend and the long-running multiplayer WebSocket process as separate deployable services.
- Added a health endpoint and Render port handling so the dual server can run as a web service.
- Runs evaluation in a separate worker so the editor remains responsive while test cases execute.

DEPLOYMENT / WHAT'S NEXT

How the pieces fit

We worked through Vercel build settings, Render service settings, and public WebSocket URLs for multiplayer rooms.

PLAYERS' BROWSER	LINK	SERVICE
Vercel frontend Vite build output: dist	<->	**JavaScript worker** Runs in the browser
Room creation, editors, challenge and scoring UI	test cases	**Executes challenge code with a two-second timeout**
Render Web Service Node WebSocket server	<->	**Room state and live code synchronization**

Deployment knowledge gained

Vercel	Build the Vite frontend with `npm run build`; the output folder is `dist`. The live frontend needs `VITE_DUAL_SERVER_URL` set to the Render WebSocket URL ending in `/dual-ws`, then a redeploy.
Render	Deploy the Node server as a Web Service from the repository root. The service starts with `npm run dual-server` and listens on Render's assigned `PORT`. The `/health` route provides a health check; the root URL can return 404 by design.
Browser evaluation	Run `npm run dev`; JavaScript challenge evaluation runs in a short-lived Web Worker without a separate runner service.

Current status and remaining dependency

Completed in the project: JavaScript challenge editing and evaluation, dual rooms, code synchronization, score comparison, and Render/Vercel configuration. The browser runs challenge code in a short-lived Web Worker; no external code runner is required.

Warlock now supports writing JavaScript solutions, checking them against challenge cases, and competing live with another player.